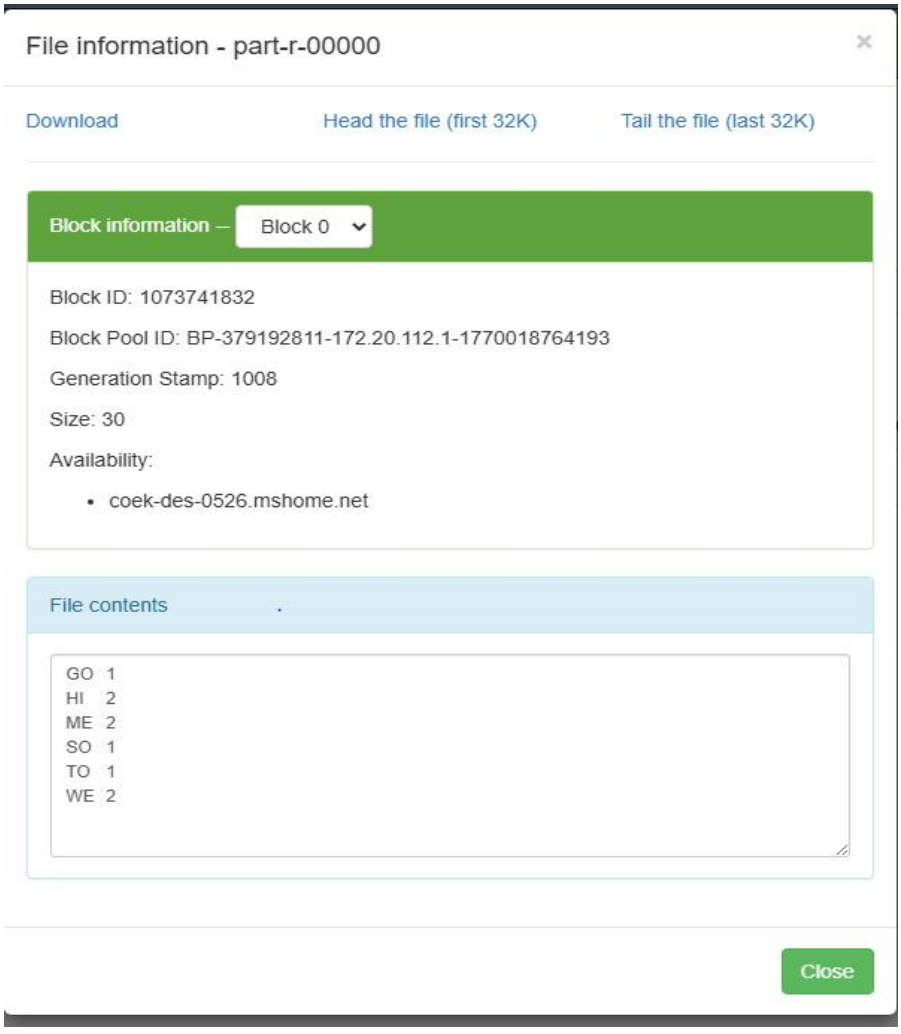
Experiment No:3

Q.Building Hadoop Map reduce application for counting frequency of word/phrase in simple text file.

```
package com.mapreduce.wc;

import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.GenericOptionsParser;

public class WordCount {
    public static void main(String[] args) throws Exception {
        Configuration c = new Configuration();
        String[] files = new GenericOptionsParser(c, args).getRemainingArgs();
        // Ensure correct input arguments
        if (files.length < 2) {
            System.err.println("Usage: WordCount <input path> <output path>");
            System.exit(-1);
        }
        Path input = new Path(files[0]);
        Path output = new Path(files[1]);

        Job j = Job.getInstance(c, "wordcount");
        j.setJarByClass(WordCount.class);
        j.setMapperClass(MapForWordCount.class);
        j.setReducerClass(ReduceForWordCount.class);
        j.setOutputKeyClass(Text.class);
        j.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(j, input);
        FileOutputFormat.setOutputPath(j, output);
        System.exit(j.waitForCompletion(true) ? 0 : 1);
    }
    // Mapper Class
    public static class MapForWordCount extends Mapper<LongWritable, Text, Text,
IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private Text wordText = new Text();
```

```
        public void map(LongWritable key, Text value, Context con) throws
IOException, InterruptedException {
            String line = value.toString().trim();
            String[] words = line.split("\\s+"); // Handles multiple spaces
            for (String word : words) {
                if (!word.isEmpty()) { // Avoid empty strings
                    wordText.set(word.trim().toUpperCase());
                    con.write(wordText, one);
                }
            }
        }
    }
    // Reducer Class
    public static class ReduceForWordCount extends Reducer<Text, IntWritable, Text,
IntWritable> {
        public void reduce(Text word, Iterable<IntWritable> values, Context con)
throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable value : values) {
                sum += value.get();
            }
            con.write(word, new IntWritable(sum));
        }
    }
}
```

Output:



Experiment No. 4

Title: Study of HADOOP YARN administrative command and user command

Command:

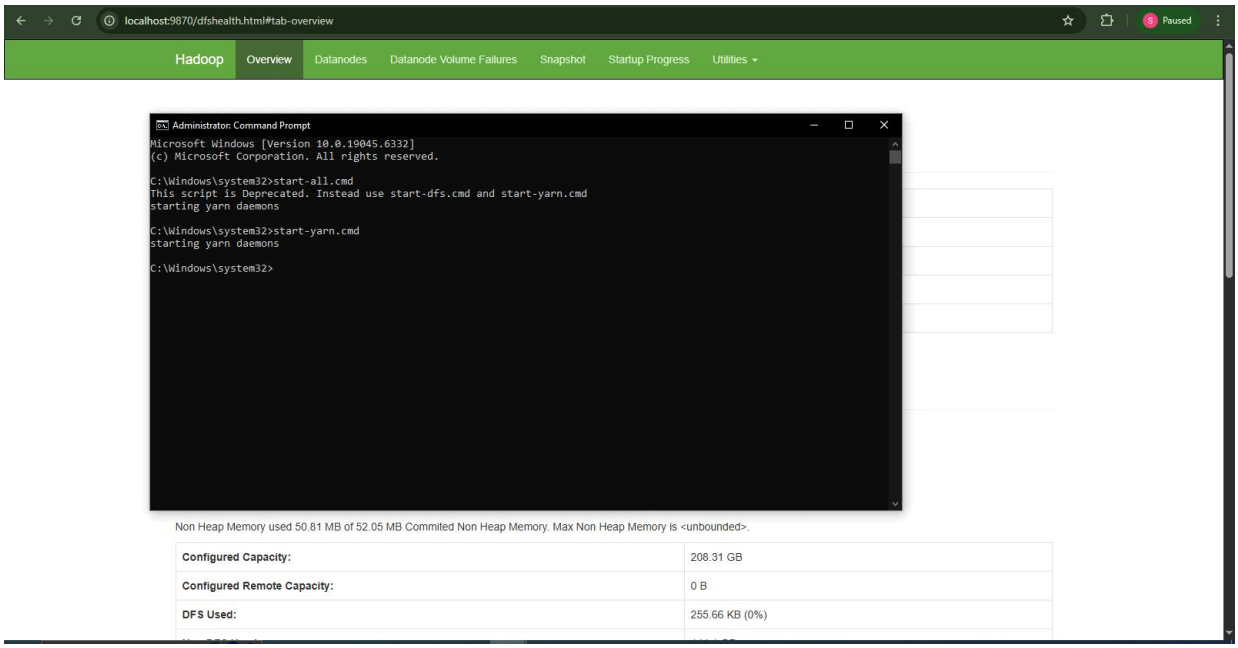
C:\Windows\system32>start-all.cmd
This script is Deprecated. Instead use start-dfs.cmd and start-yarn.cmd
starting yarn daemons

C:\Windows\system32>start-yarn.cmd
starting yarn daemons

C:\Windows\system32>stop-yarn.cmd
stopping yarn daemons
SUCCESS: Sent termination signal to the process with PID 10380.
SUCCESS: Sent termination signal to the process with PID 2628.

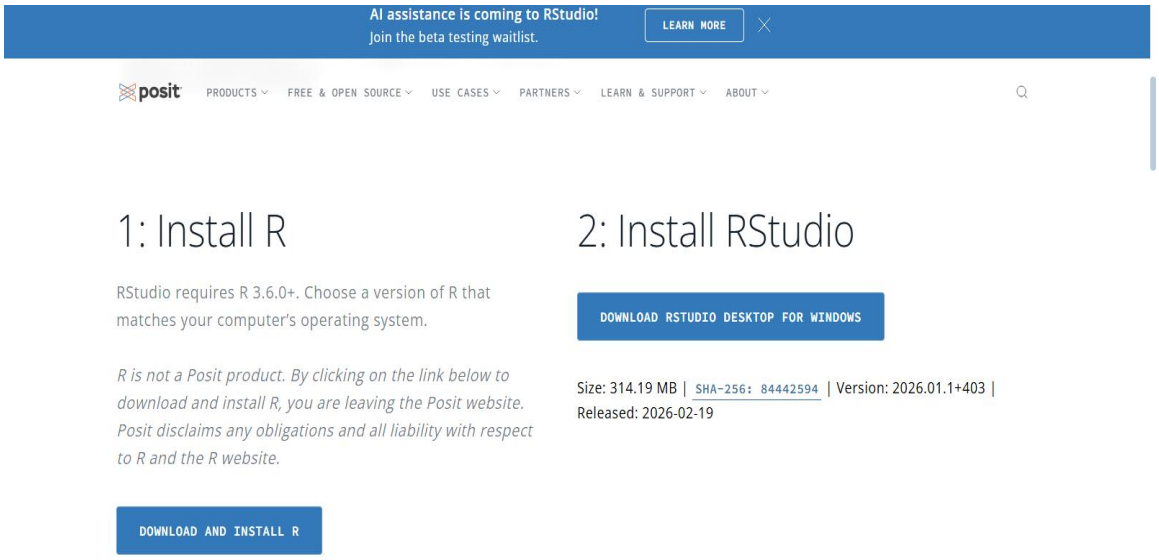
INFO: No tasks running with the specified criteria.

Output :-



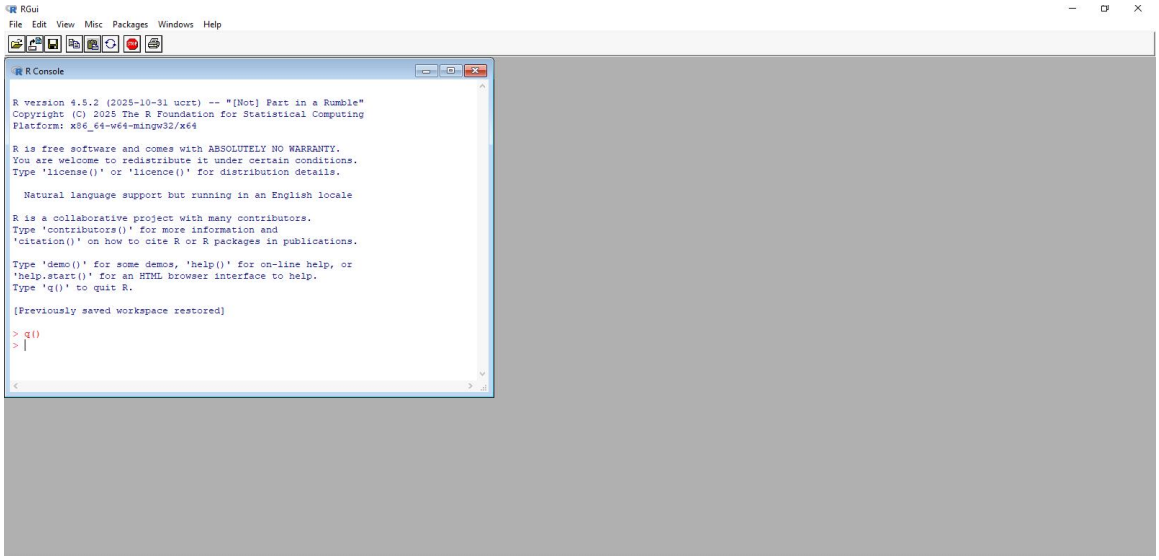
Experiment No.5

Title : Study of R and installation of R studio



RStudio :

R:


RGui
File Edit View Misc Packages Windows Help
R Console
R version 4.5.2 (2025-10-31 ucrt) -- "[Not] Part in a Rumble"
Copyright (C) 2025 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
Natural language support but running in an English locale
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
[Previously saved workspace restored]
> q()
>

Experiment No. 6

Variables:

```
name <- "John"
age <- 40
print(name)
print(age)
```

OUTPUT

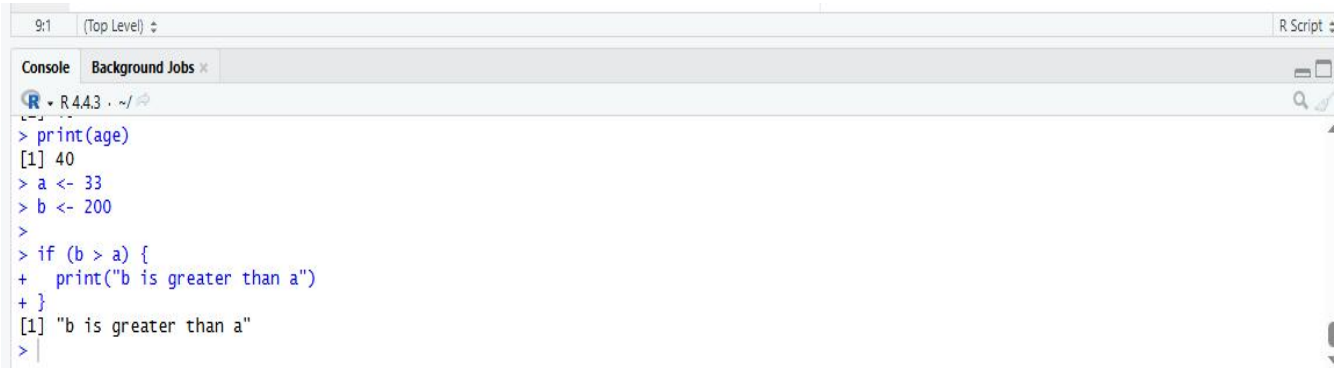
```
[1] "John"
[1] 40
> print(age)
[1] 40
>
```

Condition:

IF Statement:

```
a <- 33
b <- 200
if (b > a) {
   print("b is greater than a")
}
```

OUTPUT

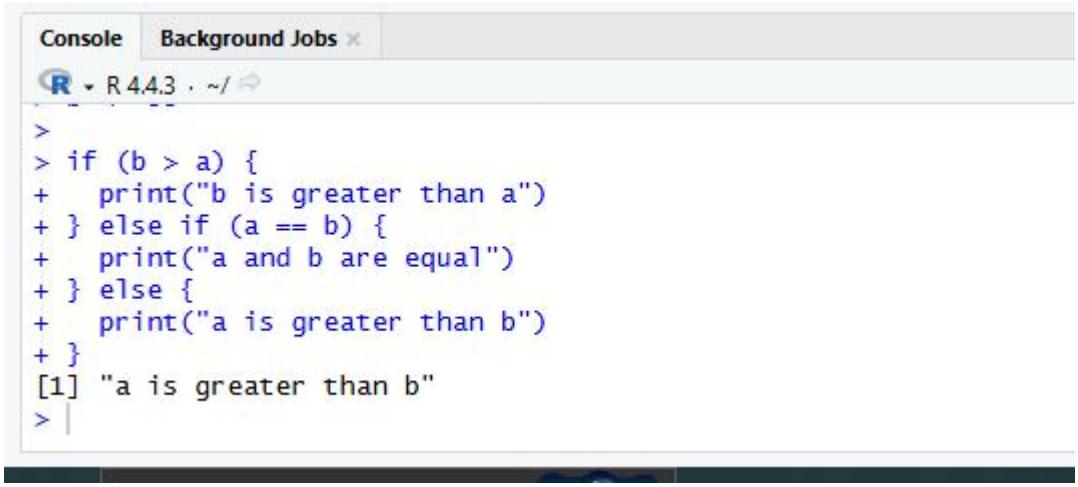


IF..ELSE Statemet:

```
a <- 200
b <- 33
if (b > a) {
   print("b is greater than a")
} else if (a == b) {
   print("a and b are equal")
```

```
} else {
   print("a is greater than b")
}
```

OUTPUT

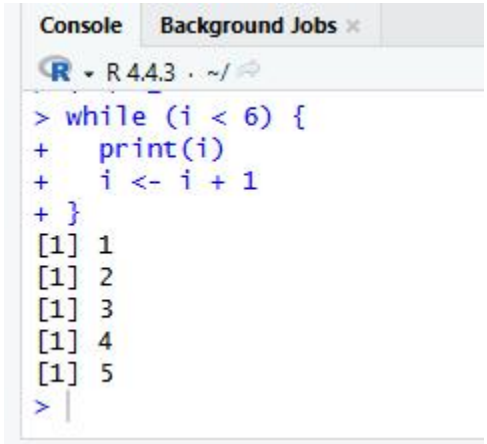


LOOPs:

While Loop

```
i <- 1
while (i < 6) {
   print(i)
   i <- i + 1
}
```

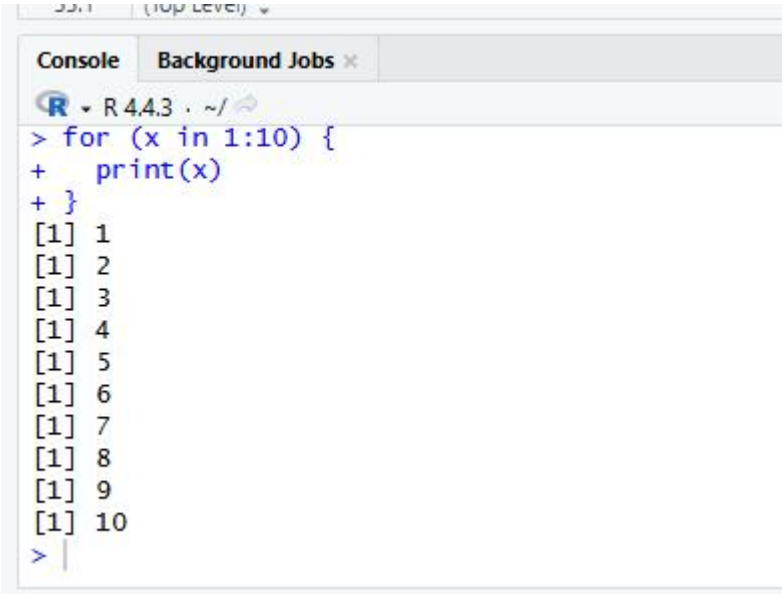
Output:



For Loop

```
for (x in 1:10) {
   print(x)
}
```

Output



Experiment No. 7

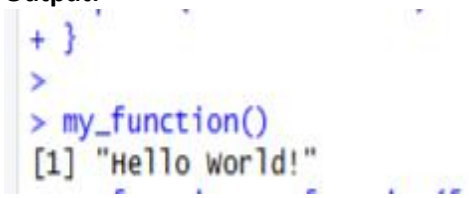
Title: To study R declaring variables,expression function amd executing R script

Functions

1. Creating a Function:

```
my_function <- function() {
  print("Hello World!")
}
```

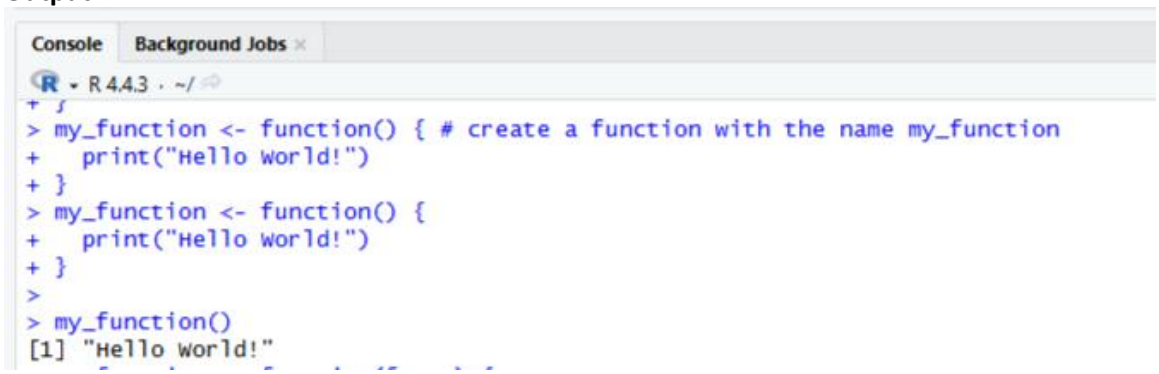
Output:



2.Call a Function

```
my_function <- function() {
  print("Hello World!")
}
my_function()
```

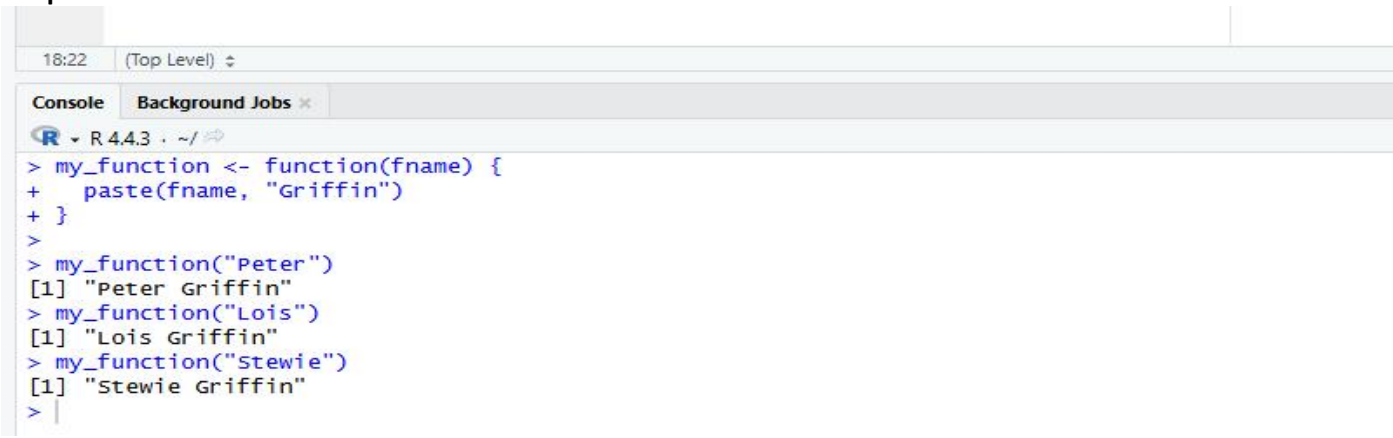
Output:



3. Arguments

```
my_function <- function(fname) {
  paste(fname, "Griffin")
}
my_function("Peter")
my_function("Lois")
my_function("Stewie")
```

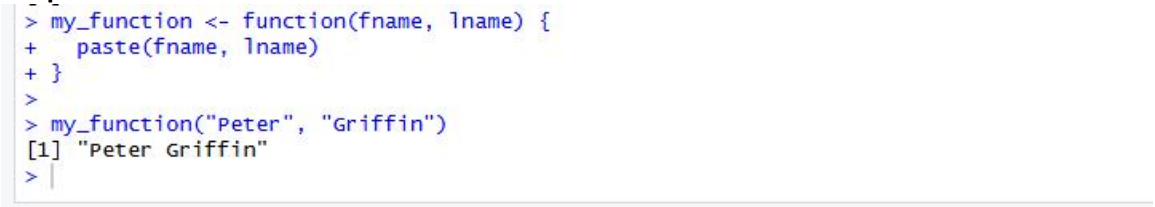
Output:



4. Number of Arguments

```
my_function <- function(fname, lname) {
  paste(fname, lname)
}
my_function("Peter", "Griffin")
```

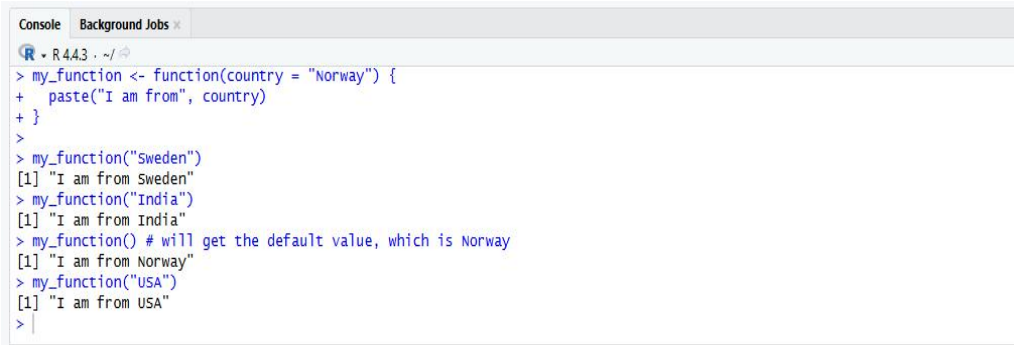
Output:



5. Default Parameter Value

```
my_function <- function(country = "Norway") {
  paste("I am from", country)
}
my_function("Sweden")
my_function("India")
my_function() # will get the default value, which is Norway
my_function("USA")
```

Output:



6. Return Values

```
my_function <- function(x) {
   return (5 * x)
}
print(my_function(3))
print(my_function(5))
print(my_function(9))
```

Output:

```
> my_function <- function(x) {
+   return (5 * x)
+ }
>
> print(my_function(3))
[1] 15
> print(my_function(5))
[1] 25
> print(my_function(9))
[1] 45
>
```

R Nested Functions:

1)
```
Nested_function <- function(x, y) {
   a <- x + y
   return(a)
}
Nested_function(Nested_function(2,2), Nested_function(3,3))
```

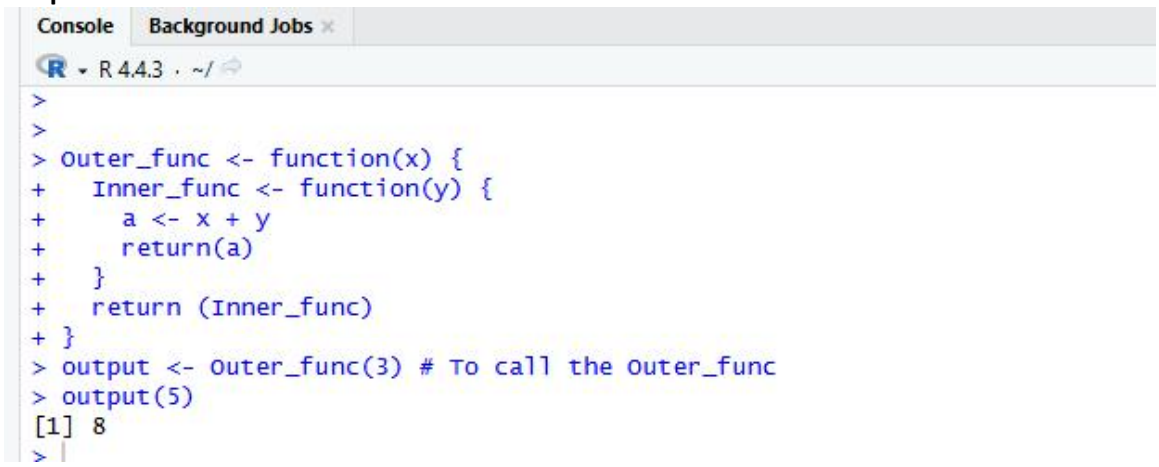
Output:

```
> Nested_function <- function(x, y) {
+   a <- x + y
+   return(a)
+ }
>
> Nested_function(Nested_function(2,2), Nested_function(3,3))
[1] 10
>
>
>
>
```

2)
```
Outer_func <- function(x) {
   Inner_func <- function(y) {
      a <- x + y
      return(a)
   }
```

```
    return (Inner_func)
}
output <- Outer_func(3) # To call the Outer_func
output(5)
```

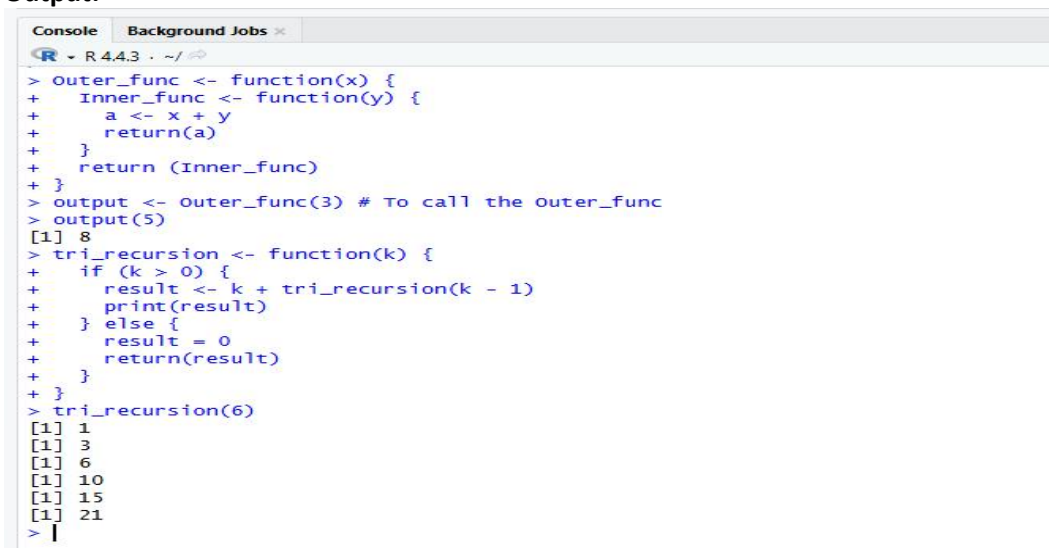
Output:



Recursion

```
tri_recursion <- function(k) {
    if (k > 0) {
        result <- k + tri_recursion(k - 1)
        print(result)
    } else {
        result = 0
        return(result)
    }
}
tri_recursion(6)
```

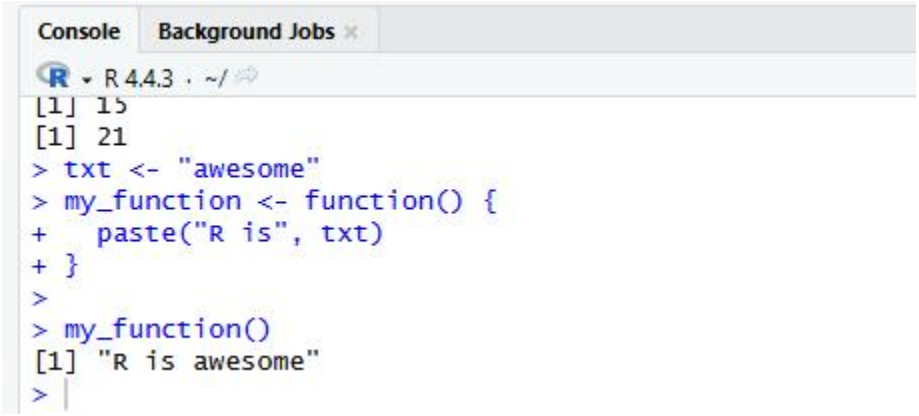
Output:



Global Variables

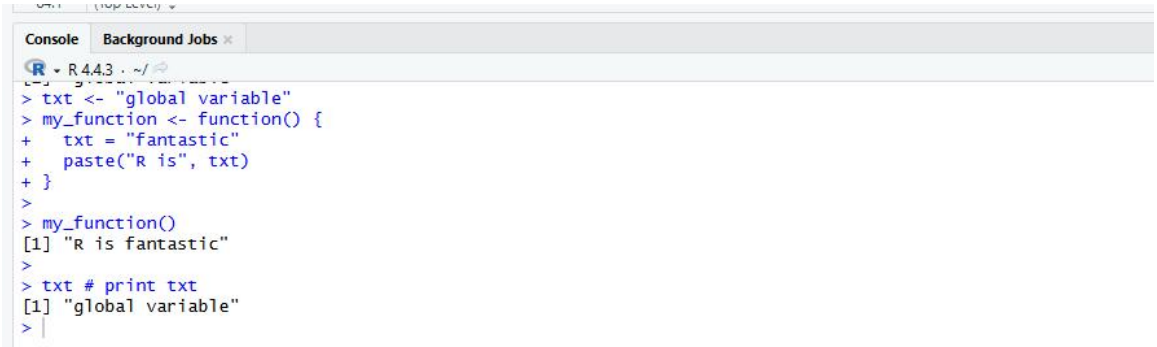
```
1) txt <- "awesome"
my_function <- function() {
   paste("R is", txt)
}
my_function()
```

Output:




```
2) txt <- "global variable"
my_function <- function() {
   txt = "fantastic"
   paste("R is", txt)
}
my_function()
txt # print txt
```

Output:



Experiment No. 8

Title: Importing data and working on dataset

Program: Data Set

```
# Print the mtcars data set
mtcars
```

Output:

```
                   mpg cyl  disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
Duster 360        14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
Merc 240D         24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
Merc 230          22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
Merc 280          19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
Merc 280C         17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
Merc 450SE        16.4   8 275.8 180 3.07 4.070 17.40  0  0    3    3
Merc 450SL        17.3   8 275.8 180 3.07 3.730 17.60  0  0    3    3
Merc 450SLC       15.2   8 275.8 180 3.07 3.780 18.00  0  0    3    3
Cadillac Fleetwood  10.4   8 472.0 205 2.93 5.250 17.98  0  0    3    4
Lincoln Continental 10.4   8 460.0 215 3.00 5.424 17.82  0  0    3    4
Chrysler Imperial   14.7   8 440.0 230 3.23 5.345 17.42  0  0    3    4
Fiat 128          32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
Honda Civic       30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
Toyota Corolla    33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
Toyota Corona     21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
Dodge Challenger  15.5   8 318.0 150 2.76 3.520 16.87  0  0    3    2
AMC Javelin       15.2   8 304.0 150 3.15 3.435 17.30  0  0    3    2
Camaro Z28        13.3   8 350.0 245 3.73 3.840 15.41  0  0    3    4
Pontiac Firebird  19.2   8 400.0 175 3.08 3.845 17.05  0  0    3    2
Fiat X1-9         27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
Porsche 914-2     26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
Lotus Europa      30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
Ford Pantera L    15.8   8 351.0 264 4.22 3.170 14.50  0  1    5    4
Ferrari Dino      19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
Maserati Bora     15.0   8 301.0 335 3.54 3.570 14.60  0  1    5    8
Volvo 142E        21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
```

Program: Get Information

```
Data_Cars <- mtcars # create a variable of the mtcars data set for better organization
# Use dim() to find the dimension of the data set
dim(Data_Cars)
# Use names() to find the names of the variables from the data set
names(Data_Cars)
```

Output:

```
[1] 32 11
 [1] "mpg"  "cyl"  "disp" "hp"   "drat" "wt"   "qsec" "vs"   "am"   "gear"
[11] "carb"
```

Program: Get Information

```
Data_Cars <- mtcars
rownames(Data_Cars)
```

Output:

```
 [1] "Mazda RX4"           "Mazda RX4 Wag"       "Datsun 710"
 [4] "Hornet 4 Drive"      "Hornet Sportabout"   "Valiant"
 [7] "Duster 360"          "Merc 240D"           "Merc 230"
[10] "Merc 280"            "Merc 280C"           "Merc 450SE"
[13] "Merc 450SL"          "Merc 450SLC"         "Cadillac Fleetwood"
[16] "Lincoln Continental" "Chrysler Imperial"   "Fiat 128"
[19] "Honda Civic"         "Toyota Corolla"      "Toyota Corona"
[22] "Dodge Challenger"    "AMC Javelin"         "Camaro Z28"
[25] "Pontiac Firebird"    "Fiat X1-9"           "Porsche 914-2"
[28] "Lotus Europa"        "Ford Pantera L"      "Ferrari Dino"
[31] "Maserati Bora"       "Volvo 142E"
```

Program: Print Variable Values

```
Data_Cars <- mtcars
Data_Cars$cyl
```

Output:

```
[1] 6 6 4 6 8 6 8 4 4 6 6 8 8 8 8 8 8 4 4 4 4 8 8 8 8 4 4 4 8 6 8 4
```

Program: Sort Variable Values

```
Data_Cars <- mtcars
sort(Data_Cars$cyl)
```

Output:

```
[1] 4 4 4 4 4 4 4 4 4 4 4 6 6 6 6 6 6 6 8 8 8 8 8 8 8 8 8 8 8 8 8 8
```

Experiment No. 10

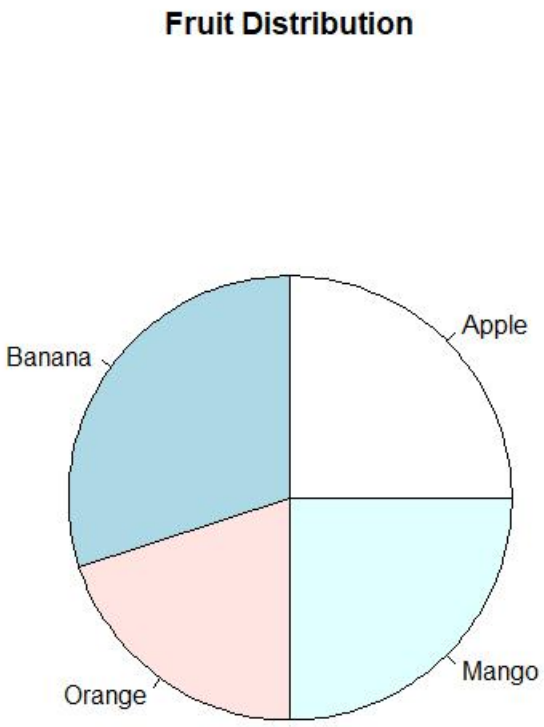
Title: Study of Data Visualization drawing Pie chart, Bar chart , Histogram, etc.

Pie Chart :

Program:

```
data <- c(25, 30, 20, 25)
labels <- c("Apple", "Banana", "Orange", "Mango")
pie(data, labels = labels, main = "Fruit")
```

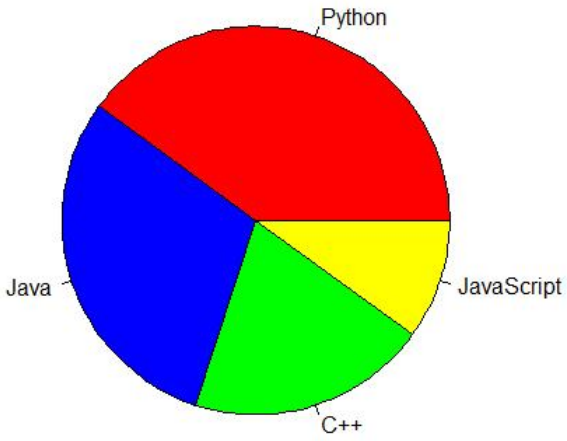
Output:-



Program:

```
data <- c(40, 30, 20, 10)
labels <- c("Python", "Java", "C++", "JavaScript")
colors <- c("red", "blue", "green", "yellow")
pie(data, labels = labels, col = colors, main = "Programming Languages")
```

Output:

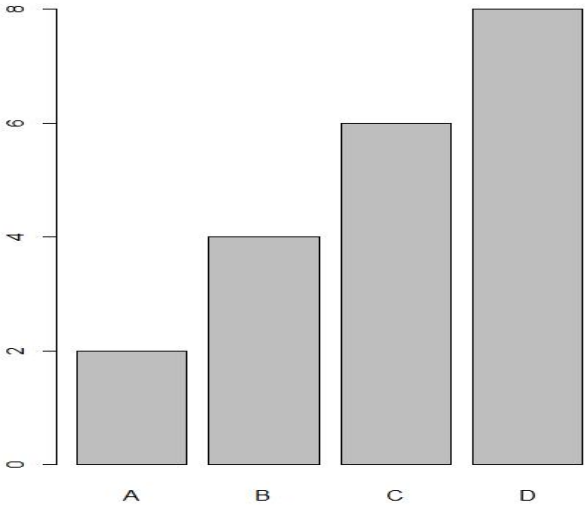


Bar Chart:

Program:

```
# x-axis values
x <- c("A", "B", "C", "D")
#y-axis values
y <- c(2, 4, 6, 8)
barplot(y, names.arg = x)
```

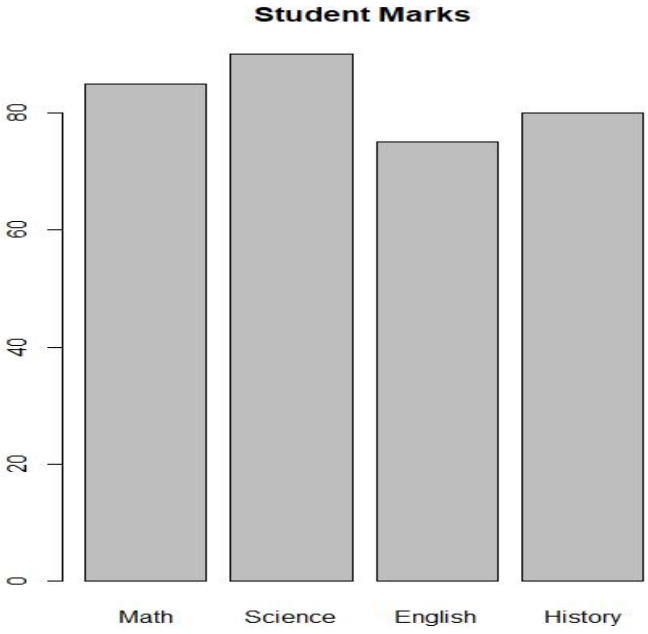
Output:



Program:

```
marks <- c(85, 90, 75, 80)
subjects <- c("Math", "Science", "English", "History")
barplot(marks, names.arg = subjects, main = "Student Marks")
```

Output

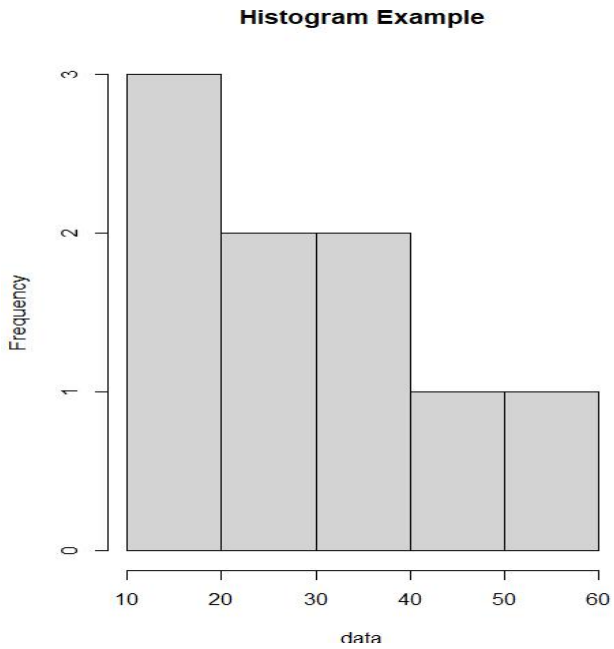


Histogram:

Program

```
data <- c(10,20,20,30,30,40,40,50,60)
hist(data, main = "Histogram Example")
```

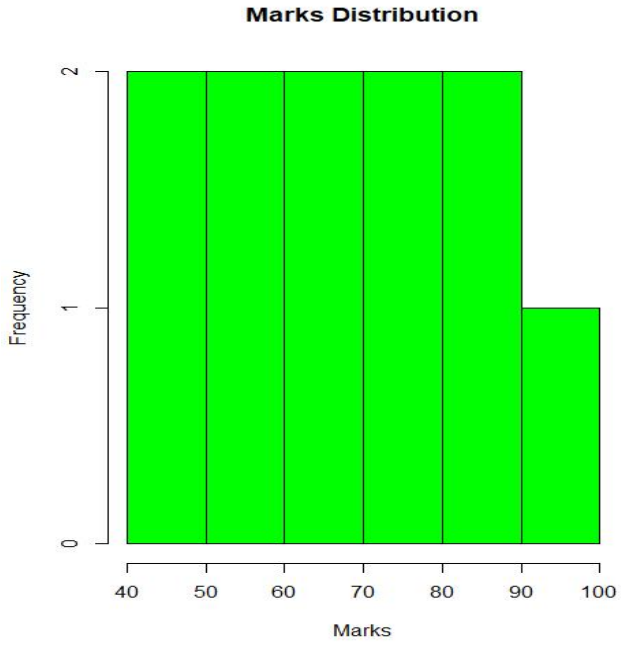
Output:



Program:

```
marks <- c(45,50,55,60,65,70,75,80,85,90,95)
hist(marks, col = "green", breaks = 5,
     main = "Marks Distribution",
     xlab = "Marks", ylab = "Frequency")
```

Output:

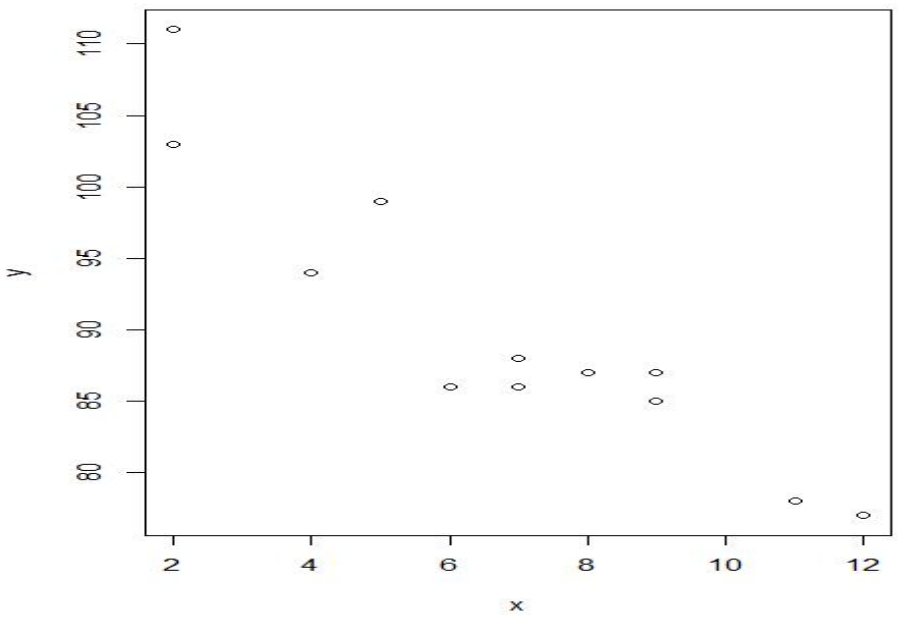


Scatter Plot:

Program

```
x <- c(5,7,8,7,2,2,9,4,11,12,9,6)
y <- c(99,86,87,88,111,103,87,94,78,77,85,86)
plot(x, y)
```

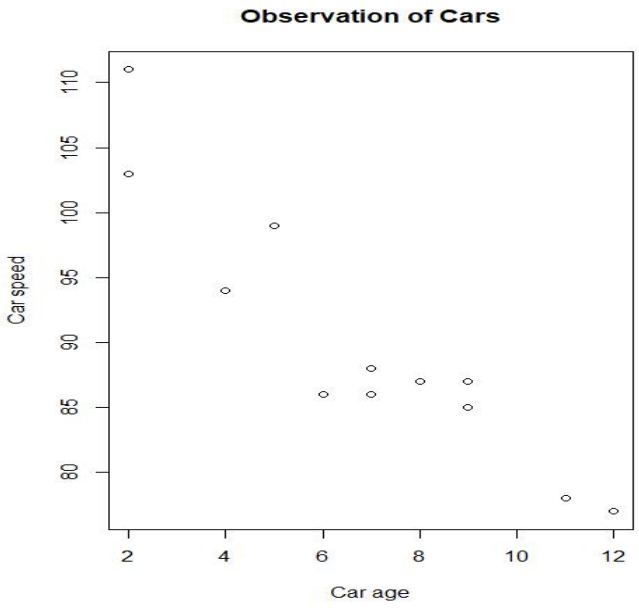
Output:



Program:

```
x <- c(5,7,8,7,2,2,9,4,11,12,9,6)
y <- c(99,86,87,88,111,103,87,94,78,77,85,86)
plot(x, y, main="Observation of Cars", xlab="Car age", ylab="Car speed")
```

Output

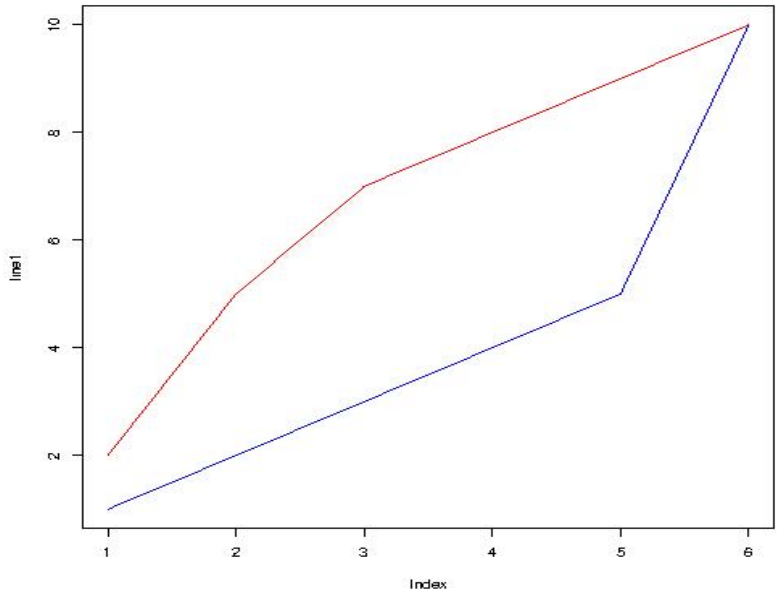


Line:

Program:

```
line1 <- c(1,2,3,4,5,10)
line2 <- c(2,5,7,8,9,10)
plot(line1, type = "l", col = "blue")
lines(line2, type="l", col = "red")
```

Output



Program:

```
plot(1:10, type="l", col="blue")
```

Output